Solution: Find if Path Exists in Graph(easy)

Problem Statement

Given an undirected graph, represented as a list of edges. Each edge is illustrated as a pair of integers `[u, v]`, signifying that there's a mutual connection between node `u` and node `v`.

Given this graph, a starting node `start`, and a destination node `end`, your task is to ascertain if a path exists between the starting node and the destination node.

Examples

1. **Example 1**:
   - Input: `4, [[0,1],[1,2],[2,3]], 0, 3`
   - Expected Output: `true`
   - Justification: There's a path from node 0 -> 1 -> 2 -> 3.
2. **Example 2**:
   - Input: `4, [[0,1],[2,3]], 0, 3`
   - Expected Output: `false`
   - Justification: Nodes 0 and 3 are not connected, so no path exists between them.
3. **Example 3**:
   - Input: `5, [[0,1],[3,4]], 0, 4`
   - Expected Output: `false`
   - Justification: Nodes 0 and 4 are not connected in any manner.

Solution

The task at hand is to determine if there's a path from a starting node to an ending node in a given undirected graph. Our approach uses Depth First Search (DFS) to explore the graph recursively. Starting at the initial node, we'll dive as deep as possible into its neighboring nodes. If we reach the target node at any point during the traversal, we know a path exists. If we exhaust all possible routes and haven't found the target, then no path exists.

1. **Graph Representation**: We'll begin by converting the provided edge list into an adjacency list to represent our graph. The adjacency list is essentially an array (or list) of lists, where each index corresponds to a node, and its content is a list of neighbors for that node. Since our graph is undirected, if there's an edge between nodes A and B, both A will be in B's list and B in A's list.
2. **Depth First Search (DFS)**: With our graph ready, we then use a recursive DFS function to traverse the graph. This function starts at the given node, and if it's the target node, we return true. Otherwise, we mark this node as visited and call the DFS function on all its unvisited neighbors. This dives deeper into the graph. If any of these recursive calls return true (meaning they found the target), our current DFS call also returns true.
3. **Handling Cycles**: To avoid getting stuck in a loop, especially in cyclic graphs, we keep track of which nodes we've visited. Before exploring a node, we'll check if it's been visited; if it has, we'll skip it.
4. **Result**: If our DFS exploration reaches the target node, we return true, signifying that a path exists. Otherwise, after checking all paths from the starting node and not finding the target, we'll conclude and return false.

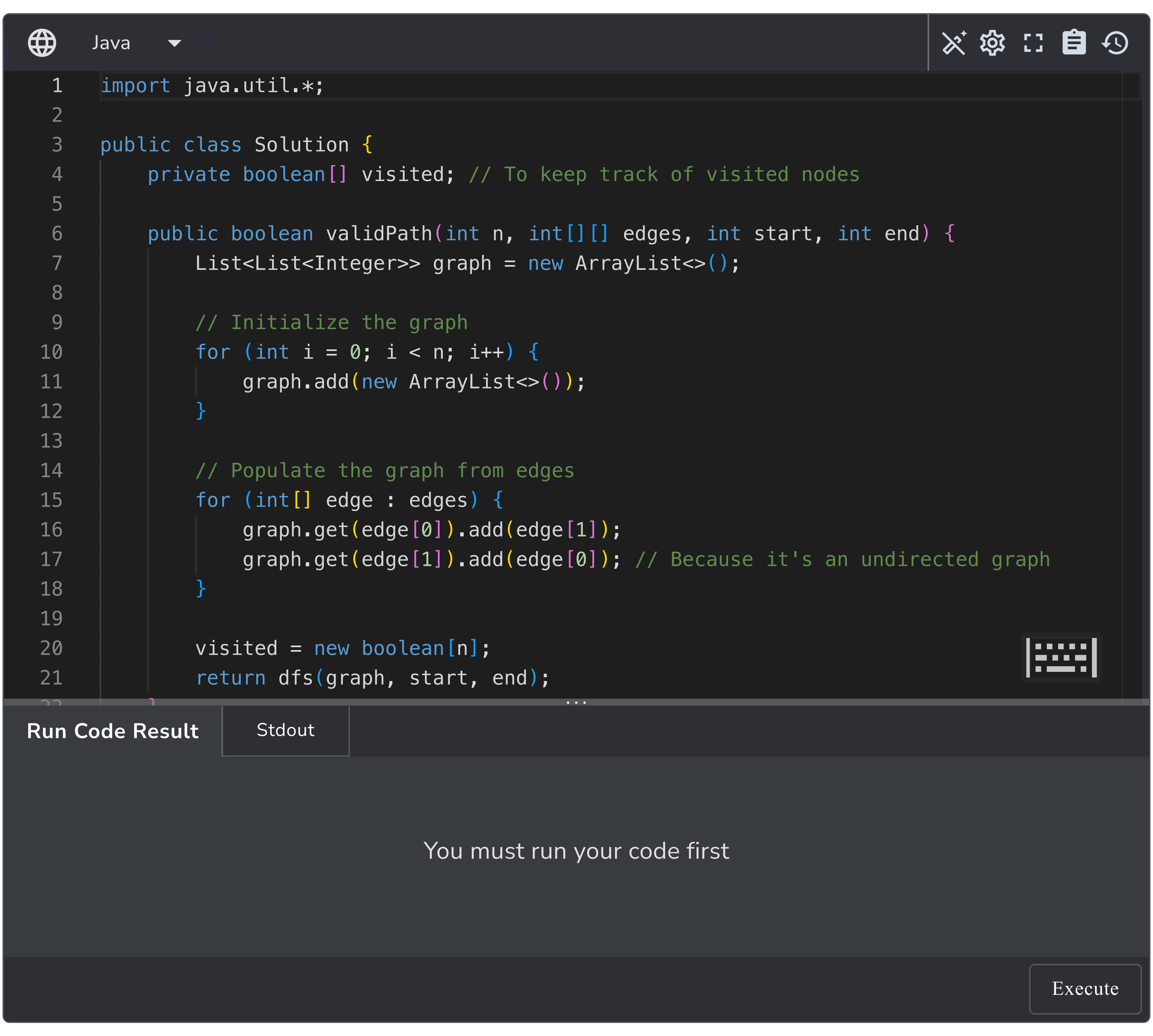
Algorithm Walkthrough: Using the input from Example 1:

- Nodes = 4, Edges = `[[0,1],[1,2],[2,3]]`, Start = 0, End = 3

1. Create the graph from the edges.
2. Begin DFS at node 0.
3. Mark node 0 as visited.
4. Explore neighbors of node 0. Discover node 1.
5. Delve into DFS for node 1.
6. Mark node 1 as visited.
7. Explore neighbors of node 1. Discover node 2.
8. Delve into DFS for node 2.
9. Mark node 2 as visited.
10. Explore neighbors of node 2. Discover node 3.
11. Since node 3 is the target `end` node, return `true`.

Code

Here is the code for this algorithm:



```java
import java.util.*;

public class Solution {
    private boolean[] visited; // To keep track of visited nodes

    public boolean validPath(int n, int[][] edges, int start, int end) {
        List<List<Integer>> graph = new ArrayList<>();

        // Initialize the graph
        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        // Populate the graph from edges
        for (int[] edge : edges) {
            graph.get(edge[0]).add(edge[1]);
            graph.get(edge[1]).add(edge[0]); // Because it's an undirected graph
        }

        visited = new boolean[n];
        return dfs(graph, start, end);
```

You must run your code first

Time Complexity

1. **Graph Construction**: Constructing the adjacency list from the given edge list takes (O(E)), where (E) is the number of edges. Each edge is processed once.
2. **DFS Traversal**: In the worst-case scenario, the Depth-First Search (DFS) can traverse all nodes and all edges once. This traversal has a time complexity of (O(V + E)), where (V) is the number of vertices or nodes, and (E) is the number of edges.

Combining the above, our time complexity is dominated by the DFS traversal, making it (O(V + E)).

Space Complexity

1. **Graph Representation**: The adjacency list requires (O(V + E)) space.
2. **Visited Set/Array**: The visited set (or array) will take (O(V)) space, as it needs to track each node in the graph.
3. **Recursive Call Stack**: The DFS function is recursive, and in the worst case (for a connected graph), it can have (V) nested calls. This would result in a call stack depth of (V), adding (O(V)) space complexity.

The dominant factor here is the graph representation and the call stack, so the total space complexity is (O(V + E)).

Summary:

- Time Complexity: (O(V + E))
- Space Complexity: (O(V + E))

This makes our algorithm efficient, especially for sparse graphs (i.e., graphs with relatively fewer edges compared to nodes). The worst-case scenario is when the graph is fully connected, but even then, our algorithm is designed to handle it within reasonable limits.

← Previous
Problem 1: Find If Path Exists In Graph(Easy)

Next →
Problem 2: Number Of Provinces (Medium)

☐ Mark as Completed